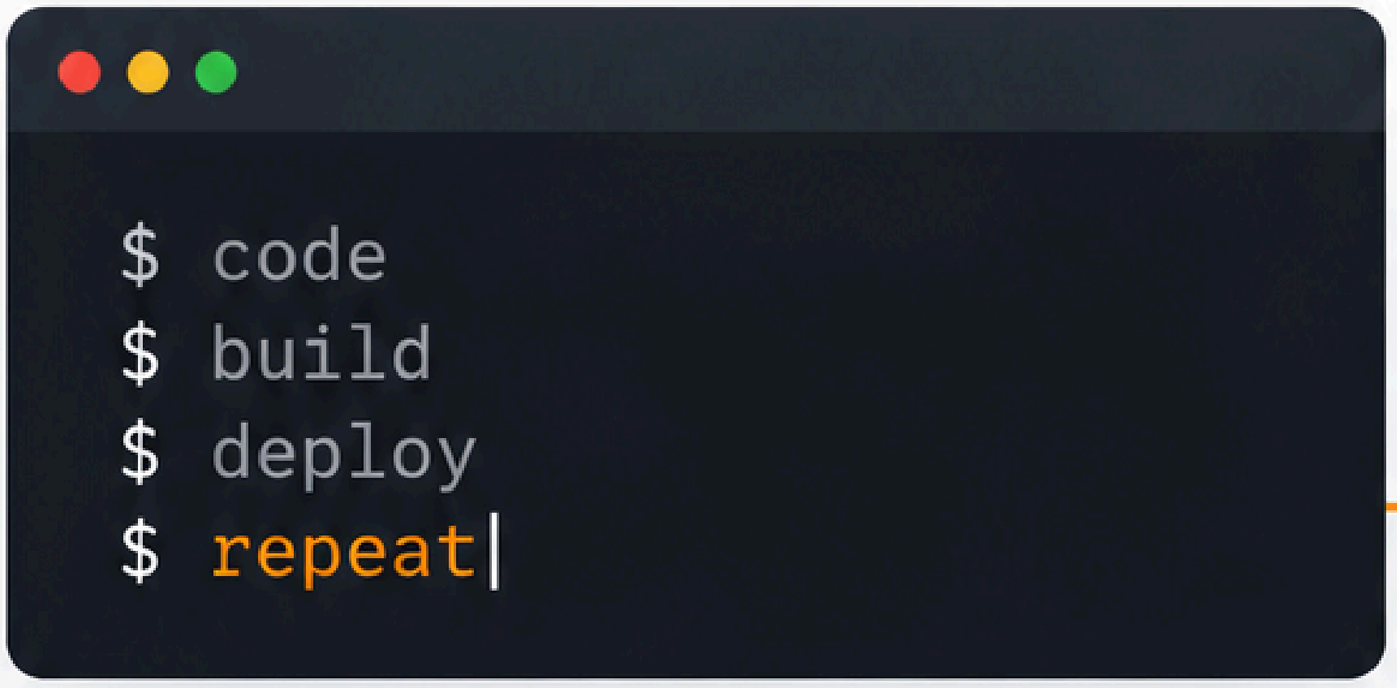
BUILD
DEPLOY
LEARN
GROW

STEP-BY-STEP GUIDE

Deploy a Containerized App on AWS

Build Today for a Bigger Tomorrow

From local setup to a scalable cloud deployment.
Hands-on. Practical. Real skills.



SAME
SKILLS.
BIGGER
POSSIBILITIES.

01	02	03	04	05	06
Ubuntu Setup	Install Docker	Install & Configure AWS CLI	AWS Configure	Amazon ECR (Private Repo)	Amazon ECS (Cluster & Deploy)

CLOUD SKILLS
REAL IMPACT

CONTAINERS
COMMUNITIES
BRIGHTER FUTURES

aws | BUILD LEARN DEPLOY GROW

Same Code Bigger Possibilities

PROJECT FLOW

From Local Setup to Cloud

A step-by-step journey to deploy a containerized Node.js app on AWS.

1

Ubuntu Setup

Set up Ubuntu, update packages and clone the project.

```
$ apt update -y
$ git clone ...
```

2

Install Docker

Install Docker Engine on Ubuntu.

```
$ apt install
  docker.io
```

3

Install & Configure AWS CLI

Install AWS CLI v2 and configure with your IAM user credentials.

```
$ curl ...
$ unzip ...
$ sudo ./aws/install
$ aws configure
```

4

AWS Configure

Configure AWS CLI with Access Key & Secret Key (from IAM user).

- AWS Access Key ID
- AWS Secret Access Key
- Default region & output format

5

Amazon ECR

Create a private repository, build and push your Docker image.

```
$ docker build ...
$ docker tag ...
$ docker push ...
```

6

Amazon ECS

Create a cluster, define a task, and deploy your containerized app.

Task Definition → ECS Cluster → Run & Deploy

Application Deployed!

Your Node.js app is now running on AWS using containers.

CLOUD SKILLS
REAL PROJECTS
BRIGHTER FUTURES

Keep Building

SETUP ENVIRONMENT

BUILD
LEARN
DEPLOY

Ubuntu & Docker

Small Steps Big Builds

Get your environment ready to clone the project and run it using Docker.

Ubuntu

1 Update Packages

Update your Ubuntu system packages.

```
apt update -y
```

2 Clone the Repository

Clone the project repository from GitHub.

```
git clone https://github.com/
Cloud-Messenger/node-todo-cicd.git
```

3 List Files

Verify the repository is cloned.

```
ls
```

4 Navigate to Project Directory

Move into the cloned repository directory.

```
cd node-todo-cicd
```

Docker Installation

5 Install Docker

Install Docker Engine on Ubuntu.

```
apt install docker.io
```

Environment Ready!

Ubuntu is updated, the project is cloned and Docker is installed.
You're all set for the next steps!

Let's Build Something!

CODE • CONTAIN • DEPLOY

AWS CLI

3 Install AWS CLI

Remove the old version (if any) and install the latest AWS CLI on Ubuntu.

Latest Tools Brighter Builds

AWS CLI

24 Remove Existing AWS CLI

Uninstall any previously installed version.

```
sudo apt remove awscli
```

25 Download AWS CLI v2

Download the latest AWS CLI installer.

```
curl "https://awscli.amazonaws.com/awscli-exe-linux-x86_64.zip" -o "awscliv2.zip"
```

26 Unzip the Installer

Extract the downloaded file.

```
unzip awscliv2.zip
```

27 Install AWS CLI

Run the installer.

```
sudo ./aws/install
```

AWS CLI Installed!

You are now ready to configure AWS CLI.

One step closer to the cloud!

AWS CLI

4 Configure AWS CLI

Set up your AWS credentials to connect to your AWS account.

Secure Access Real Possibilities

1 Run Configure Command

Start the AWS CLI configuration.

```
aws configure
```

2 Enter Credentials

You will be prompted to enter:

- **AWS Access Key ID**
- AWS Secret Access Key
- Default region name (e.g., ap-south-1)
- Default output format (e.g., json)

Where to get Access Key & Secret Key?

1. Go to AWS Console → IAM → Users.
2. Select your IAM user (e.g., Tanay).
3. Go to the Security credentials tab.
4. Click **Create access key** and copy the **Access Key ID** and **Secret Access Key**.

3 Configuration Complete

Once entered, your AWS CLI will be configured and ready to use.

AWS CLI Configured!

You can now access your AWS resources from the command line.

AWS ECR

5 Create a Public ECR Repository

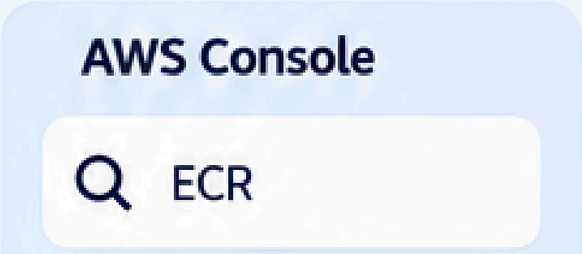
BUILD
STORE
DEPLOY

Create a public repository in Amazon ECR to store and push your Docker image.

1 Open Amazon ECR

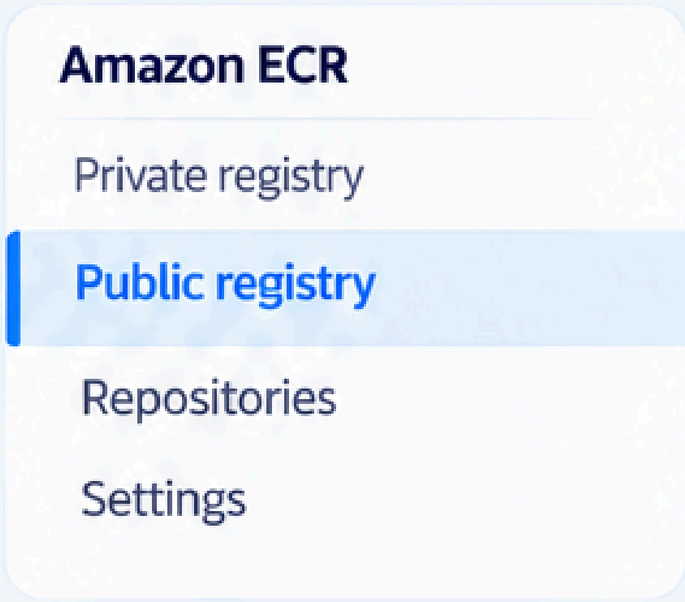
In the AWS Management Console, search for **"ECR"** and open **Amazon ECR**.





2 Create Public Repository

In the left navigation pane, click **Public registry → Repositories → Create repository.**



3 Repository Created

Your public ECR repository is now ready.
Note down the repository URI (you'll use it to tag and push your Docker image).



Next

Your public ECR repository is ready!

You can now push your Docker image to this repository using the repository URI.

One Step Closer

CONTAINERS
ENABLE
POSSIBILITIES

aws

AMAZON ECS

6 Create ECS Cluster, Task & Run

Deploy your container using Amazon ECS (without a load balancer).

BUILD
CONTAINERS
DEPLOY
REAL IMPACT

From Image to Application

1 Create ECS Cluster

- Open Amazon ECS → Create cluster.
- Choose the cluster type (e.g., Fargate).
- **Enable monitoring** (Container Insights).
- Give a name to your cluster and **create** it.

Key Settings

- Cluster name: (e.g., tanay-cluster)
- Infrastructure: AWS Fargate
- Monitoring: ☑ Enable (Container Insights)

2 Create Task Definition

- Go to Task Definitions → Create new task definition.
- Choose launch type: **Fargate**.
- Add container using your **ECR image** (e.g., public.ecr.aws/<alias>/tanay:latest).
- Set container port to 8000.
- For **Execution Role**, create a new **IAM role** and attach the required policies (listed on the right).

Create IAM Role & Attach Policies

Create a new role for Task Execution and attach these 6 AWS managed policies:

1. AmazonECSTaskExecutionRolePolicy
2. AmazonElasticContainerRegistryPublicFullAccess
3. AmazonElasticContainerRegistryPublicPowerUser
4. AmazonElasticContainerRegistryPublicReadOnly
5. EC2InstanceProfileForImageBuilder
6. EC2InstanceProfileForImageBuilderECRContainerBuilds

3 Run the Task

- Go to your cluster → **Run task**.
- Select the task definition you created.
- Choose launch type: **Fargate**.
- Set the task count (e.g., 1) and run the task.

Run Task Settings

- Launch type: Fargate
- Task definition: (your created task)
- Task count: 1

Run task

4 Configure Security Group

- After the task is running, open the security group attached to the task (or the one created).
- In inbound rules, add the following:
  - Port 80 (Custom TCP)
  - Port 8000 (Custom TCP)
  - Source: Anywhere (0.0.0.0/0)

Inbound Rules

Type	Port range	Source
Custom TCP	80	0.0.0.0/0
Custom TCP	8000	0.0.0.0/0

5 Access the Application

- Go to the running task (or task details).
- Copy the public IP of the running task.
- Paste the IP in your browser to open the application.

Example

http://<public-ip>

Your application is live!

Your container is now running on Amazon ECS and accessible via its public IP.

Small Steps Big Deployments

aws | CONTAINERS • CLOUD • OPPORTUNITIES

DEPLOY TODAY
BUILD TOMORROW